
LOAN APPROVAL PREDICTION SYSTEM

Project Id: AIML-8

Name : Bhavini Vats

Class Roll Number : 09

University Roll Number : 2028408

Section and Group: K-G1

Name : Aakriti Chauhan

Class Roll Number : 04

University Roll Number : 2028374

Section and Group: K-G1

TABLE OF CONTENTS

1. Problem Description.....	3
1.1. Dataset Description.....	3
1.2. Flow diagram.....	5
2. Modules for Project Implementation.....	6
2.1. Modules Used.....	6
2.2. Platform Used.....	8
3. Machine Learning model/ Searching Technique used.....	9
4. Evaluation metrics used.....	11
5. Results and Visualizations.....	12
6. Conclusion and Future Scope.....	13
7. References.....	15

1. Problem Description

The Loan Approval Prediction System is a machine learning-based project developed to predict whether a loan application should be approved or rejected, using applicant attributes such as credit score, annual income, debt-to-income ratio, employment status, and other financial data. The system uses data preprocessing, feature engineering, and machine learning algorithms to analyse the dataset and generate accurate predictions. It helps financial institutions identify high-risk applicants, automate the loan evaluation process, and support better credit decision-making.

In the traditional banking sector, loan approval is a manual and time-intensive process. Loan officers must review each application individually, assess the applicant's financial background, and make a judgment call that is inherently subjective. This process becomes increasingly difficult to scale as the number of applications grows, leading to delays and inconsistency in decision-making. Moreover, human bias can inadvertently influence approval decisions, raising fairness concerns.

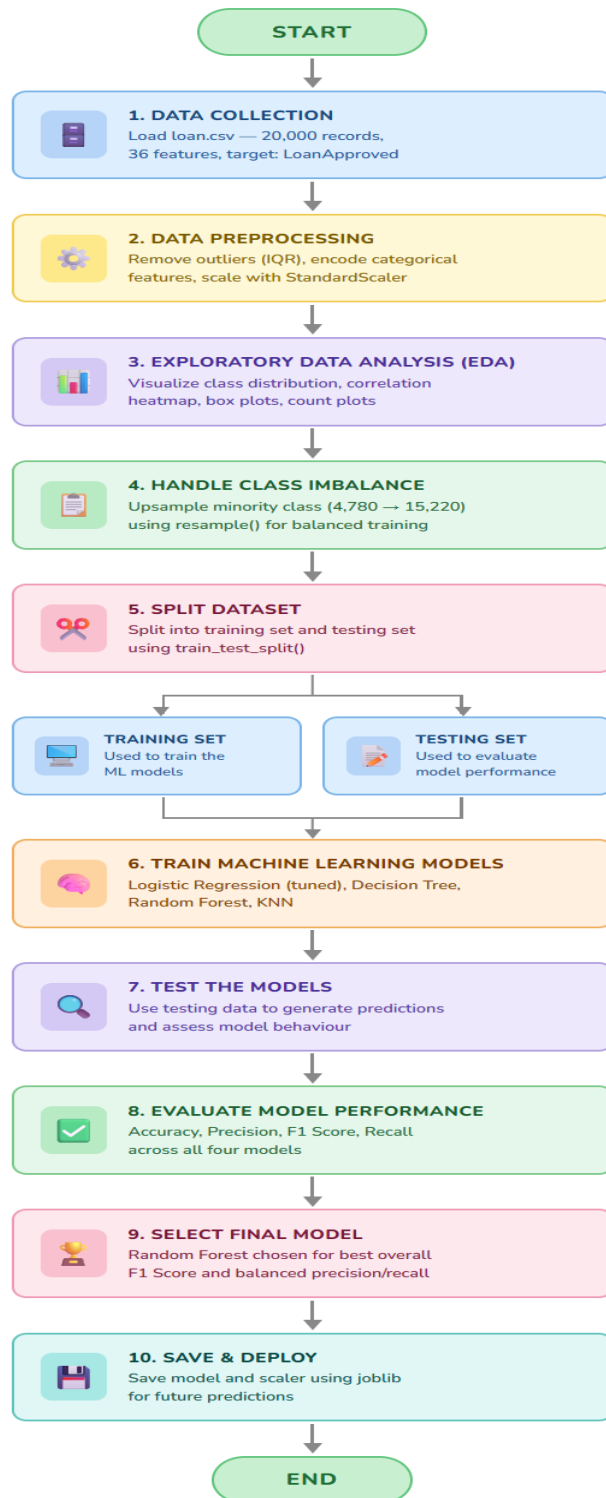
1.1 Dataset Description

The dataset used in this project (loan.csv) contains information related to applicant financial profiles and loan details. It includes 36 features such as Age, AnnualIncome, CreditScore, EmploymentStatus, EducationLevel, LoanAmount, LoanDuration, MaritalStatus, DebtToIncomeRatio, PreviousLoanDefaults, PaymentHistory, NetWorth, and more. The dataset contains 20,000 records.

The dataset was loaded in CSV format and processed using Python libraries. Data preprocessing techniques such as outlier removal using the IQR method, data augmentation to address class imbalance (upsampling the minority class), and feature scaling using StandardScaler were applied before training the machine learning models.

The target variable of the dataset is LoanApproved (binary: 0 = Rejected, 1 = Approved). The original dataset was imbalanced with 15,220 rejected and 4,780 approved cases, which was addressed through upsampling to balance the classes to 15,220 each before model training.

1.2 Flow Diagram



2. Modules for Project Implementation

2.1. Modules Used

1. Pandas

Pandas is used for data manipulation and analysis. It provides the DataFrame structure which makes it easy to load, inspect, filter, and transform tabular data. In this project, Pandas is used to read the dataset, explore its structure, separate majority and minority classes for balancing, and perform column-level operations during preprocessing.

Functions Used:

- `pd.read_csv()` – Reads the dataset from CSV file into a DataFrame.
- `DataFrame.head()` – Displays the first few rows of the dataset to verify successful loading.
- `DataFrame.info()` – Shows column names, data types, and non-null counts to identify missing values and incorrect types.
- `DataFrame.describe()` – Generates statistical summaries (mean, std, min, max, quartiles) for all numerical features.
- `DataFrame.value_counts()` – Counts occurrences of each class in the target variable to assess class imbalance.

2. NumPy

NumPy is used for numerical computations. It provides efficient array operations and mathematical functions that underpin many operations in Pandas and Scikit-learn. In this project, NumPy is primarily used during the data augmentation step where small random perturbations are added to income values of upsampled records to create more realistic synthetic data.

Functions Used:

- `np.array()` – Creates numerical arrays for batch operations.
- `np.mean()` – Calculates the mean of feature values during statistical analysis.
- `np.std()` – Calculates the standard deviation to understand feature spread.

3. Matplotlib

Matplotlib is the foundational plotting library used for data visualization. It provides full control over plot elements such as axes, titles, labels, and figure size. All visualizations in this project — whether generated directly with Matplotlib or via Seaborn — are ultimately rendered through Matplotlib's engine.

Functions Used:

- `plt.plot()` – Draws line graphs for trend visualization.
- `plt.bar()` – Creates bar charts for comparing model accuracies or feature distributions.
- `plt.show()` – Renders and displays the current figure in the output.

4. Seaborn

Seaborn is a high-level visualization library built on Matplotlib. It provides cleaner, more informative statistical plots with less code. It was used extensively during the EDA phase to understand feature distributions, class imbalance, and inter-feature correlations.

Functions Used:

- `sns.heatmap()` – Displays a colour-coded correlation matrix of all numerical features, helping identify which features are most strongly associated with the target variable `LoanApproved`.
- `sns.countplot()` – Shows the count of each class in the `LoanApproved` column, used to visualize class imbalance before and after upsampling.

5. Scikit-learn

Scikit-learn is the primary machine learning library used in this project. It provides consistent APIs for data preprocessing, model training, hyperparameter tuning, and performance evaluation. All four classification models are implemented using Scikit-learn's estimator interface.

Functions Used:

- `train_test_split()` – Splits the balanced dataset into 80% training and 20% testing sets with a fixed random seed for reproducibility.
- `LogisticRegression()` – Builds a logistic regression classification model.
- `DecisionTreeClassifier()` – Creates a decision tree model based on feature thresholds.
- `accuracy_score()` – Calculates the proportion of correct predictions made by the model.
- `RandomForestClassifier()` – Builds an ensemble random forest model selected as the final model.

- `KNeighborsClassifier()` – Implements K-Nearest Neighbors classification.
- `GridSearchCV()` – Performs exhaustive hyperparameter tuning with cross-validation for Logistic Regression.
- `StandardScaler()` – Normalizes feature values to zero mean and unit variance before model training.

2.2 Platform Used

Visual Studio Code: Visual Studio Code was used for writing, testing, and debugging the Python code. It provides syntax highlighting, extensions, and debugging tools which simplify development. Its integrated terminal allows running Python scripts and Jupyter Notebooks side by side, making it a convenient all-in-one development environment.

Jupyter Notebook: Jupyter Notebook was used for data analysis, visualization, and model training. It allows execution of code in separate cells and helps in easy visualization of outputs and graphs. The cell-based structure is particularly useful for iterative data exploration, as individual preprocessing steps and model evaluations can be run and inspected independently without re-executing the entire script.

Python: Python programming language was used because of its simplicity and strong support for machine learning and data analysis libraries. Its clean syntax reduces development time, and its large ecosystem of open-source libraries — including Pandas, NumPy, Matplotlib, Seaborn, and Scikit-learn — makes it the industry standard for machine learning projects.

3. Machine Learning model/ Searching Technique used

Four machine learning classification models were trained and evaluated in this project. Each model approaches the loan approval classification problem differently, offering a variety of learning strategies, interpretability levels, and performance trade-offs. The goal was to identify the model that provides the best balance of accuracy, precision, recall, and F1 Score for this task.

1. Logistic Regression

Logistic Regression is used for binary classification tasks. It models the probability that an applicant will receive loan approval based on input features using the sigmoid (logistic) function, which maps any real-valued input to a probability between 0 and 1. If the predicted probability exceeds 0.5, the applicant is classified as approved; otherwise, they are classified as rejected. Unlike linear regression, logistic regression is specifically designed for classification and does not assume a linear relationship between the features and the output — only between the features and the log-odds of the output.

Advantages:

- Efficient and fast to train, even on large datasets
- Provides probability scores for classification, useful for setting custom thresholds
- Works well for linearly separable data

2. Decision Tree

Decision Tree is a supervised learning algorithm used for binary classification of loan approval. It builds a tree-like structure by recursively splitting the dataset at each node based on the feature and threshold that best separates the classes, using criteria such as Gini impurity or information gain. The splitting continues until the tree reaches a specified depth or all data points at a node belong to the same class. Each leaf node of the tree represents a final classification — either approved or rejected.

Advantages:

- Easy to understand and visualize without requiring domain expertise
- Handles both numerical and categorical data
- Requires less preprocessing compared to other algorithms

3. K-Nearest Neighbors (KNN)

KNN predicts whether a loan is approved or rejected based on the k nearest data points in feature space. When a new application is submitted, the algorithm calculates the distance between the new data point and all training data points, selects the k closest ones, and assigns the class that appears most frequently among them. It is a non-parametric algorithm, meaning it makes no assumptions about the underlying distribution of the data.

Advantages:

- Simple algorithm with no explicit training phase
- Effective for small datasets
- No training phase required, making it quick to set up

4. Random Forest (Final Model)

Random Forest is an ensemble learning algorithm that constructs a large number of decision trees during training and outputs the class that receives the majority of votes across all individual trees. Each tree is trained on a randomly sampled subset of the training data (with replacement), and at each node, only a random subset of features is considered for splitting. These two sources of randomness reduce the correlation between individual trees, significantly lowering the overall variance of the ensemble compared to a single decision tree.

Advantages:

- Highly resistant to overfitting due to ensemble averaging across many trees
- Handles high-dimensional feature spaces effectively
- Works well on imbalanced datasets
- Provides feature importance scores that can be used to understand which factors drive loan approval decisions

4. Evaluation metrics used

To assess and compare the performance of each machine learning model, the following standard classification evaluation metrics were used. Since the dataset originally contained a significant class imbalance — far more rejected cases than approved ones — accuracy alone is not a reliable indicator of model quality. A model that simply predicts 'Rejected' for every application would still achieve around 76% accuracy without learning anything meaningful. Therefore, Precision, Recall, and F1 Score were equally important in this evaluation.

1. Accuracy

Accuracy measures the overall proportion of predictions — both approved and rejected — that were correct out of all predictions made by the model.

$$\text{Accuracy} = (TP + TN) / (TP + TN + FP + FN)$$

Where TP = True Positive (correctly predicted approved), TN = True Negative (correctly predicted rejected), FP = False Positive (predicted approved but actually rejected), FN = False Negative (predicted rejected but actually approved). While useful as a general measure, accuracy can be misleading in imbalanced scenarios and must be interpreted alongside Precision and Recall.

2. Precision

Precision measures the proportion of applicants predicted as approved who were actually approved. It answers the question: of all the applicants the model approved, how many truly deserved approval?

$$\text{Precision} = TP / (TP + FP)$$

High precision means fewer false positives — fewer applicants who should have been rejected are being mistakenly approved. From a lender's perspective, this is critical because approving a loan for someone who will default leads to direct financial loss. A model with high precision is conservative and cautious in its approvals.

3. F1 Score and Recall

Recall measures the proportion of all actual approved applicants that the model correctly identified as approved. It answers the question: of all applicants who truly deserved approval, how many did the model catch?

$$\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$$

High recall means fewer false negatives — fewer creditworthy applicants are being wrongly rejected. From a business perspective, this matters because rejecting good applicants means lost revenue and a poor customer experience. The F1 Score is the harmonic mean of Precision and Recall, combining both into a single balanced metric.

$$\text{F1 Score} = 2 * (\text{Precision} * \text{Recall}) / (\text{Precision} + \text{Recall})$$

The F1 Score is particularly useful when the class distribution is uneven, as in this project. A model with high precision but very low recall — or vice versa — will have a low F1 Score, signalling that its performance is one-sided. F1 Score was used as the primary criterion for final model selection, alongside overall accuracy. Random Forest was chosen as the final model because it achieved the most balanced combination of F1 Score and precision-recall trade-off among all four models tested.

5. Results and Visualizations

Four machine learning models were trained and tested using the loan dataset: Logistic Regression, Decision Tree, Random Forest, and KNN. Data balancing via upsampling was applied to address class imbalance. Different visualizations such as count plots and box plots were generated for analysis. Features used: CreditScore, AnnualIncome, and DebtToIncomeRatio.

The box plots identified and removed outliers in AnnualIncome using the IQR method. The first box plot clearly showed extreme values in the upper range of the AnnualIncome distribution. After applying IQR-based clipping, a second box plot confirmed that the outliers had been successfully removed, resulting in a much cleaner and more representative income distribution.

A count plot of the LoanApproved column before balancing revealed a significant class imbalance — 15,220 rejected cases versus only 4,780 approved cases. After upsampling the minority class using sklearn's resample() function, a second count plot confirmed that both classes were equalized at 15,220 records each. This balanced dataset was used for all subsequent model training.

A correlation heatmap was generated to visualize pairwise relationships between all numerical features. The heatmap revealed that CreditScore, AnnualIncome, and DebtToIncomeRatio had the most meaningful correlation with the target variable LoanApproved, which informed the feature selection for model training.

Model performance comparison: Logistic Regression achieved 84.1% accuracy (best overall accuracy), Random Forest 82.8%, KNN 81.4%, and Decision Tree 76.9%. However, when considering the balance between precision and recall as captured by the F1 Score, Random Forest demonstrated the most consistent and reliable performance. Its ensemble nature made it more robust to overfitting than the Decision Tree, and its recall was higher than Logistic Regression, meaning it missed fewer creditworthy applicants.

Random Forest was selected as the final model due to its balanced precision and recall, and its overall robustness across the test dataset. The system successfully predicts loan approval outcomes. The trained Random Forest model and scaler were saved using joblib for deployment. The model can help financial institutions reduce manual evaluation time and improve credit decision accuracy.

6. Conclusion and Future Scope

Conclusion

The Loan Approval Prediction System successfully applies machine learning techniques to predict loan approval outcomes based on applicant financial attributes. The project automates credit assessment tasks, reduces manual analysis, and improves decision-making efficiency for financial institutions.

The implementation of multiple machine learning models, data balancing techniques, and visualization tools helped identify the most important features affecting loan approval decisions such as credit score, annual income, and debt-to-income ratio.

Throughout the development of this project, several important insights were gained. Addressing class imbalance before training is essential — without upsampling, models would predict rejections for nearly all cases and perform poorly on approved applicants. Feature scaling is a critical preprocessing step, especially for distance-based algorithms like KNN that are sensitive to the magnitude of input values. Evaluating models using multiple metrics beyond accuracy alone provides a far more honest picture of real-world performance. The comparison of four models demonstrated that model selection should be driven by the specific performance requirements of the deployment context — in this case, a balance of precision and recall was prioritized over raw accuracy.

The final Random Forest model was serialized along with the StandardScaler using joblib, making the system ready for integration into a real banking application where new applicant data can be evaluated in real time without retraining.

Future Scope

- Integration with real-time banking and credit bureau APIs for live data ingestion at the time of application
- Deployment as a web application using Flask or FastAPI with a user-friendly interface for bank staff and applicants

- Use of advanced models like XGBoost and neural networks to further improve prediction accuracy
- Addition of more financial features such as credit history length, savings balance, and number of existing loans
- Improved prediction accuracy using larger and more geographically diverse datasets to reduce bias
- Implementation of SHAP (SHapley Additive exPlanations) values to provide transparent, interpretable reasons for each approval or rejection decision
- Hyperparameter tuning for Random Forest and KNN using GridSearchCV or Bayesian Optimization

7. References

Books: Let Us Python – Yashavant Kanetkar

Online Resources:

- GeeksforGeeks – Machine Learning and Python articles
- TutorialsPoint – Python and ML reference guides
- Stack Overflow – Programming Q&A community
- Scikit-learn Documentation – <https://scikit-learn.org/stable/documentation.html>
- Pandas Documentation – <https://pandas.pydata.org/docs/>
- Matplotlib Documentation – <https://matplotlib.org/stable/contents.html>
- Seaborn Documentation – <https://seaborn.pydata.org/>
- NumPy Documentation – <https://numpy.org/doc/>
- Towards Data Science (Medium) – Articles on ML model evaluation and feature engineering
- Kaggle – Dataset references and community notebooks on loan prediction